



**SCHOOL OF  
COMPUTING**

# **Design Analysis of Algorithm**

**CSE211**

**Week-3**

**CH.SC.U4CSE24221**

**TEJESH KANTA**

## BFS

### Code:

```
C bfs.c > [?]n
1  #include <stdio.h>
2
3  #define MAX 10
4
5  int queue[MAX], front = -1, rear = -1;
6  int visited[MAX] = {0};
7  int adj[MAX][MAX];
8  int n;|
9
10 void enqueue(int v) {
11     if (rear == MAX - 1)
12         return;
13     if (front == -1)
14         front = 0;
15     queue[++rear] = v;
16 }
17
18 int dequeue() {
19     if (front == -1 || front > rear)
20         return -1;
21     return queue[front++];
22 }
23
24 void bfs(int start) {
25     int i, v;
26     enqueue(start);
27     visited[start] = 1;
28
29     while (front <= rear) {
30         v = dequeue();
31         printf("%d ", v);
32
33         for (i = 0; i < n; i++) {
34             if (adj[v][i] == 1 && visited[i] == 0) {
35                 enqueue(i);
36                 visited[i] = 1;
37             }
38         }
39     }
40 }
```

## BFS

### Code:

```
37     }
38 }
39 }
40 }
41
42 int main() {
43     int i, j, start;
44
45     printf("CH.U4CSE24221-KANTA TEJESH\n");
46
47     printf("Enter number of vertices: ");
48     scanf("%d", &n);
49
50     printf("Enter adjacency matrix:\n");
51     for (i = 0; i < n; i++)
52         for (j = 0; j < n; j++)
53             scanf("%d", &adj[i][j]);
54
55     printf("Enter starting vertex: ");
56     scanf("%d", &start);
57
58     printf("BFS Traversal: ");
59     bfs(start);
60
61     return 0;
62 }
63
```

### OUTPUT:

```
[kantatejesh@KANTAs-MacBook-Air ~ % cd Desktop
[kantatejesh@KANTAs-MacBook-Air Desktop % cd Website
[kantatejesh@KANTAs-MacBook-Air Website % gcc bfs.c -o bfs
[kantatejesh@KANTAs-MacBook-Air Website % ./bfs
CH.U4CSE24221-KANTA TEJESH
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
Enter starting vertex: 1
BFS Traversal: 1 0 3 2 4 %
```

## BFS

*Code:*

### *Space Complexity and it's Justification:*

Time Complexity

- $O(V^2)$

*Justification:*

- Uses adjacency matrix
- For each vertex, all V vertices are checked

Space Complexity

- $O(V)$

*Justification:*

- Queue array + visited array store at most V vertices

## DFS

### Code:

```
C dfs.c > ...
1  #include <stdio.h>
2
3  #define MAX 10
4
5  int visited[MAX] = {0};
6  int adj[MAX][MAX];
7  int n;
8
9  void dfs(int v) {
10     int i;
11     printf("%d ", v);
12     visited[v] = 1;
13
14     for (i = 0; i < n; i++) {
15         if (adj[v][i] == 1 && visited[i] == 0) {
16             dfs(i);
17         }
18     }
19 }
20
21 int main() {
22     int i, j, start;
23
24     printf("CH.U4CSE24221-KANTA TEJESH\n");
25
26     printf("Enter number of vertices: ");
27     scanf("%d", &n);
28
29     printf("Enter adjacency matrix:\n");
30     for (i = 0; i < n; i++)
31         for (j = 0; j < n; j++)
32             scanf("%d", &adj[i][j]);
33
34     printf("Enter starting vertex: ");
35     scanf("%d", &start);
36
37     printf("DFS Traversal: ");
38     dfs(start);
39
40     return 0;
41 }
```

### OUTPUT:

## DFS

### Code:

```
kantatejesh@KANTAs-MacBook-Air Website % ./dfs
CH.U4CSE24221-KANTA TEJESH
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
Enter starting vertex: 1
DFS Traversal: 1 0 2 4 3 %
```

### Space Complexity and it's Justification:

#### Time Complexity

- $O(V^2)$

Justification:

- Adjacency matrix representation
- Each DFS call scans all vertices

#### Space Complexity

- $O(V)$

Justification:

- Recursive call stack + visited array